

Tech Challenge - Pipeline Híbrida de Alfabetização

Repositório do projeto: https://github.com/techposrc-ops/tech_challenger_fase_2

Tech Challenge - Pipeline Híbrido de Alfabetização

Projeto desenvolvido para o Tech Challenge da Fase 2. A solução implementa uma pipeline híbrida de dados educacionais no Google Cloud, combinando processamento Batch e Streaming com Arquitetura Medalhão, qualidade de dados, observabilidade e práticas de FinOps.

Contexto do problema

A alfabetização na infância influencia toda a trajetória escolar. O Compromisso Nacional Criança Alfabetizada busca garantir que as crianças estejam alfabetizadas ao final do 2º ano do ensino fundamental. Para acompanhar esse objetivo, o Indicador Criança Alfabetizada considera alfabetizado o estudante que alcança o nível de proficiência definido pelo INEP.

Uma análise confiável não pode observar apenas um indicador isolado. Por isso, este projeto integra metas nacionais, estaduais e municipais, dados territoriais, resultados educacionais e microdados de alunos. A base resultante permite comparar metas e resultados, acompanhar a evolução temporal e identificar desigualdades entre territórios.

Objetivo

Construir uma pipeline escalável e reproduzível que:

- ingira dados históricos e eventos em tempo quase real;
- preserve os dados brutos e seu histórico;
- limpe, padronize e valide as informações;
- integre as seis entidades educacionais;
- disponibilize indicadores analíticos por Brasil, UF e município;
- registre execução, duração, volume processado, rejeições e falhas;
- utilize recursos de nuvem com controle de custo.

Fonte de dados

Os dados são públicos e pertencem ao conjunto [Avaliação da Alfabetização](#), disponibilizado pelo INEP por meio da Base dos Dados. A pipeline integra as tabelas do dataset `basedosdados.br_inep_avalicao_alfabetizacao`:

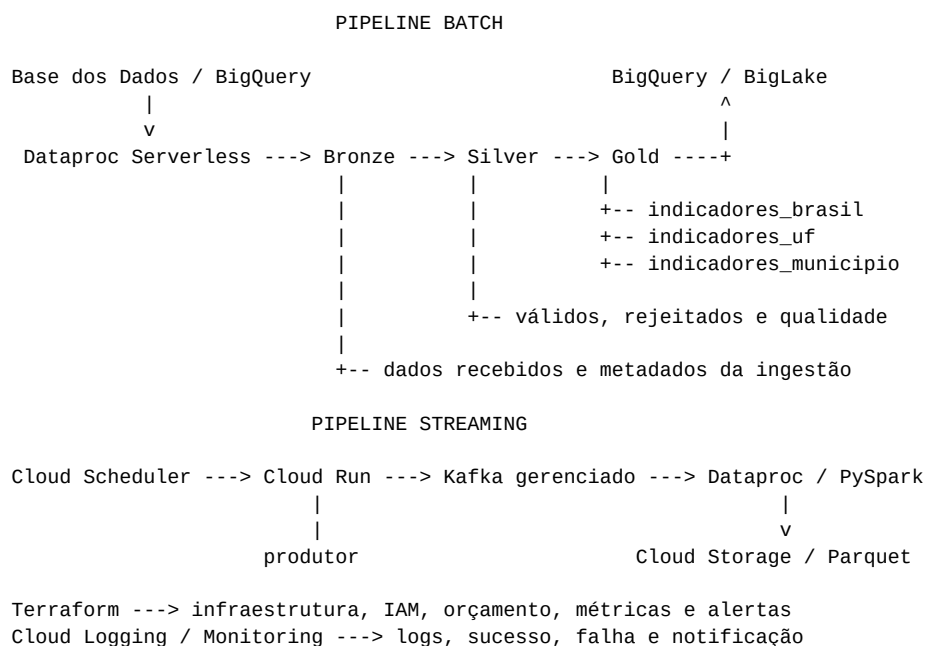
Tabela	Conteúdo
<code>uf</code>	Indicadores de desempenho por unidade federativa
<code>meta_alfabetizacao_brasil</code>	Resultados e metas nacionais
<code>meta_alfabetizacao_uf</code>	Metas estaduais
<code>meta_alfabetizacao_municipio</code>	Metas municipais

Tabela	Conteúdo
município	Indicadores territoriais municipais
alunos	Microdados educacionais

O desenvolvimento do projeto começou localmente com os arquivos CSV originais baixados do INEP e armazenados em `data/arquivos_raw`. Esses arquivos foram usados na exploração inicial dos dados, na construção dos notebooks e na criação e validação das camadas Bronze, Silver e Gold antes da implantação em nuvem. Eles permanecem versionados para permitir a reprodução acadêmica local.

Na GCP, os CSVs locais não são utilizados: a Bronze lê diretamente as seis tabelas públicas da Base dos Dados pelo conector do BigQuery.

Arquitetura da solução



Fluxo Batch

1. A Bronze lê as seis fontes do BigQuery na nuvem ou os CSVs no ambiente local.
2. Cada carga recebe identificador, data de ingestão, origem e nome da tabela.
3. A Silver lê a carga mais recente, converte tipos, limpa textos e remove duplicidades.
4. Registros com chaves nulas ou percentuais fora de 0 a 100 são separados como rejeitados.
5. Os relacionamentos entre alunos, municípios e UFs são validados antes da Gold.
6. A Gold integra resultados, metas e dados agregados dos alunos.
7. As três tabelas Gold são disponibilizadas para consulta analítica no BigQuery.

Fluxo Streaming

1. O Cloud Scheduler aciona o produtor executado no Cloud Run.
2. O produtor publica eventos de atualização no Managed Service for Apache Kafka.
3. O consumidor PySpark Structured Streaming processa os eventos no Dataproc.
4. O job aplica esquema explícito, deduplicação por `event_id`, watermark e checkpoint.
5. Os eventos processados são gravados em Parquet no Cloud Storage.

O fluxo Streaming foi validado ponta a ponta com 10 eventos publicados no Kafka, consumidos pelo PySpark e gravados na Bronze de Streaming. Os recursos temporários foram desativados após o teste.

Arquitetura Medalhão

Bronze - dados brutos

- preserva os valores recebidos sem transformações de negócio;
- mantém o histórico por `id_ingestao`;
- registra data, arquivo ou tabela de origem e identificador da carga;
- grava em Parquet no ambiente local ou no Cloud Storage.

Silver - dados tratados

- padroniza textos e tipos numéricos;
- elimina duplicidades pelas chaves de negócio;
- verifica chaves obrigatórias e valores ausentes;
- valida percentuais no intervalo de 0 a 100;
- separa registros válidos e rejeitados com o motivo da rejeição;
- valida a relação entre alunos, municípios e UFs;
- produz um relatório de qualidade por execução.

Erros críticos interrompem a construção da Gold, evitando a publicação de uma camada analítica inconsistente.

Gold - dados analíticos

Produto	Finalidade
<code>indicadores_brasil</code>	Evolução nacional e comparação com as metas de 2024 a 2030
<code>indicadores_uf</code>	Taxa oficial, proficiência, participação e situação da meta por UF
<code>indicadores_municipio</code>	Resultado oficial, microdados agregados e situação da meta municipal

As tabelas calculam `meta_do_ano`, `diferenca_para_meta` e `situacao_meta`. A Gold municipal também apresenta alunos presentes, alunos alfabetizados, taxa calculada pelos microdados e média de proficiência. Identificadores individuais não são publicados nessa camada.

Tecnologias utilizadas

Tecnologia	Uso e justificativa
Python 3.11	Linguagem principal, simples e adequada ao objetivo acadêmico
PySpark	Processamento distribuído comum aos fluxos Batch e Streaming
Pandas e Jupyter	Exploração inicial e apresentação didática dos dados
Parquet	Formato colunar, comprimido e eficiente para leitura analítica

Tecnologia	Uso e justificativa
BigQuery	Acesso à Base dos Dados e consulta das tabelas Gold
Cloud Storage	Data Lake das camadas Bronze, Silver e Gold
Dataproc Serverless	Execução Batch sem manter cluster ocioso
Apache Kafka gerenciado	Transporte de eventos com interface Kafka padrão
Cloud Run	Execução do produtor com escala mínima igual a zero
Cloud Scheduler e Workflows	Agendamento e coordenação dos serviços
Cloud Logging e Monitoring	Logs estruturados, métricas e alertas
Terraform	Infraestrutura reproduzível, versionada e configurável

Decisões arquiteturais e trade-offs

Batch e Streaming

O Batch é usado para as seis tabelas históricas, pois elas são atualizadas periodicamente e exigem integração completa. O Streaming simula novas medições e atualizações em tempo quase real. Manter os dois fluxos aumenta a complexidade, mas demonstra como tratar cargas completas e eventos sem acoplar as regras a uma única forma de ingestão.

Data Lake e Data Warehouse

O Cloud Storage guarda Parquet nas camadas Medalhão com baixo custo e preserva o histórico. O BigQuery fornece acesso SQL para análise e dashboards. Essa combinação evita usar o Data Warehouse como área de arquivos brutos e evita duplicar desnecessariamente todos os dados processados.

Custo e desempenho

A região us-central1 foi escolhida pelo custo e pela disponibilidade dos serviços usados. O Batch roda no Dataproc Serverless e o Cloud Run escala para zero. Kafka e o cluster do consumidor são opcionais e ficam desativados por padrão, pois melhoram a experiência de streaming, mas geram custo enquanto provisionados.

Portabilidade

As regras ficam em `src/alfabetizacao`, enquanto os adaptadores de implantação ficam em `cloud/gcp`. Os caminhos locais ou `gs://`, a origem e o projeto de faturamento são parâmetros. O uso de Python, Spark, Kafka e Parquet reduz o esforço necessário para adaptar a solução a outra nuvem.

Qualidade e governança de dados

As validações executadas incluem:

- presença das seis tabelas esperadas;
- duplicidade pelas chaves de negócio;
- nulidade nas chaves obrigatórias;
- percentuais menores que 0 ou maiores que 100;
- existência do município relacionado ao aluno;
- existência da UF relacionada ao município;
- existência de registros válidos antes da Gold;
- armazenamento dos rejeitados e do relatório de qualidade.

Cada execução recebe um identificador único. Credenciais, variáveis locais, estados Terraform, arquivos brutos e resultados processados permanecem fora do controle de versão.

Observabilidade

Os pipelines produzem logs estruturados em JSON com:

- início e fim da execução;
- identificador e status;
- duração total e por camada;
- quantidade de registros processados;
- quantidade de registros rejeitados;
- mensagem de erro em caso de falha.

O Terraform cria métricas de sucesso e falha no Cloud Logging e uma política no Cloud Monitoring. Quando configurado, o canal de e-mail recebe alertas de falha dos pipelines. Exemplos de consultas e campos estão em docs/observabilidade.md.

FinOps

As principais decisões de controle de custo são:

- Parquet e organização das camadas no mesmo Data Lake;
- seleção apenas das colunas necessárias e filtro na origem;
- Dataproc Serverless para evitar cluster Batch ocioso;
- Cloud Run com escala mínima igual a zero;
- Kafka, consumidor e orquestração Streaming ativados separadamente;
- expiração de checkpoints após sete dias;
- mudança da Bronze para Nearline após trinta dias;
- orçamento mensal com alertas em 50%, 90% e 100%;
- desligamento dos recursos temporários depois das validações.

O teste Batch consumiu 2.002.650 milliDCU-seconds e 202.800 GiB-seconds de shuffle, com custo estimado em **US\$ 0,0365**. Considerando também uma hora de Kafka e meia hora do Dataproc Streaming, o teste completo foi estimado em aproximadamente **US\$ 0,43**, sem impostos, câmbio, rede e operações de armazenamento. As fórmulas estão em docs/finops.md e podem ser recalculadas com:

```
.\.venv\Scripts\python.exe scripts\estimar_custos.py
```

Os preços são parâmetros da estimativa e devem ser revisados na página oficial da GCP antes de uma nova execução.

Potencial de aplicação em IA

A Gold foi criada sem identificadores individuais e pode alimentar:

- modelos de previsão da taxa de alfabetização por município;
- identificação de territórios com risco de não atingir as metas;
- agrupamento de municípios por desempenho e vulnerabilidade;
- análise de desigualdade educacional entre regiões;
- avaliação do efeito de políticas públicas ao longo do tempo;

- priorização de recursos e ações de apoio pedagógico.

Antes de um uso preditivo, recomenda-se enriquecer os indicadores com dados socioeconômicos, estrutura escolar, território e financiamento, além de avaliar viés, explicabilidade e qualidade das variáveis.

Estrutura do repositório

cloud/gcp/	implantação e execução dos serviços GCP
data/bronze/	destino local dos dados brutos
data/silver/	destino local dos dados tratados e rejeitados
data/gold/	destino local dos indicadores analíticos
docs/	arquitetura, dados, observabilidade e FinOps
infrastructure/terraform/	infraestrutura GCP como código
notebooks/	exploração e construção didática das camadas
scripts/	utilitários do projeto
src/alfabetizacao/batch/	pipeline Bronze, Silver, Gold e orquestração
src/alfabetizacao/streaming/	produtor, simulador e consumidor PySpark
.env.example	exemplo de configuração local
pyproject.toml	dependências e empacotamento Python

Os dados gerados em data/bronze, data/silver e data/gold não são versionados. As pastas representam a organização lógica do Data Lake e são preenchidas pela execução da pipeline.

Preparação do ambiente local

Requisitos: Python 3.11, Java 17 e Git.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -e "[batch]"
```

Instale apenas os grupos necessários para cada atividade:

```
# Qualidade e verificações locais
python -m pip install -e "[dev]"

# Jupyter, gráficos e PySpark
python -m pip install -e "[notebooks]"

# Produtor Kafka e consumidor Streaming
python -m pip install -e "[streaming]"
```

Selecione o interpretador .venv como kernel dos notebooks.

PySpark no Windows

No Windows nativo, o Hadoop usado pelo PySpark precisa encontrar winutils.exe para gravar arquivos Parquet. Instale uma distribuição Hadoop compatível, deixe o executável em C:\hadoop\bin\winutils.exe e configure a sessão antes de iniciar o pipeline:

```
$env:HADOOP_HOME="C:\hadoop"
$env:Path="$env:HADOOP_HOME\bin;$env:Path"
$env:PYSARK_PYTHON=(Resolve-Path ".\venv\Scripts\python.exe").Path
$env:PYSARK_DRIVER_PYTHON=$env:PYSARK_PYTHON
Test-Path "$env:HADOOP_HOME\bin\winutils.exe"
```

O último comando deve retornar True. Como alternativa, execute o projeto em Linux ou WSL, onde o winutils.exe não é necessário. Essa configuração é usada somente no desenvolvimento local; os jobs do Dataproc já executam em ambiente Linux gerenciado pela GCP.

Execução local

Coloque os seis arquivos originais em data/arquivos_raw e execute, na raiz do projeto:

```
$env:PYTHONPATH="src"
python -m alfabetizacao.batch.orquestracao `
  --ambiente local `
  --pasta-raw data/arquivos_raw `
  --pasta-base data
```

Para abrir os notebooks:

```
python -m jupyter lab
```

Os notebooks apresentam a evolução didática do projeto: exploração inicial e construção das camadas Bronze, Silver e Gold.

Implantação na GCP

Execução automatizada

O script `scripts/executar_gcp.ps1` reúne as verificações, o Terraform e os comandos de execução. Primeiro, autentique sua própria conta:

```
gcloud auth login
gcloud auth application-default login
```

Para apenas gerar e revisar o plano, sem criar recursos:

```
.\scripts\executar_gcp.ps1 `
  -Projeto "SEU_PROJECT_ID" `
  -Etapa planejar
```

Para criar a infraestrutura, executar Bronze, Silver e Gold e registrar as tabelas no BigQuery:

```
.\scripts\executar_gcp.ps1 `
  -Projeto "SEU_PROJECT_ID" `
  -Etapa batch `
  -ConfirmarCustos
```

Depois de validar o Batch, o teste completo de Streaming pode ser executado com:

```
.\scripts\executar_gcp.ps1 `
  -Projeto "SEU_PROJECT_ID" `
  -Etapa streaming `
  -ConfirmarCustos
```

Ao terminar a demonstração, remova Kafka, Cloud Run, Scheduler, Workflows e o cluster Dataproc:

```
.\scripts\executar_gcp.ps1 `
  -Projeto "SEU_PROJECT_ID" `
  -Etapa desligar-streaming `
  -ConfirmarCustos
```

O bucket do estado usa, por padrão, o nome `SEU_PROJECT_ID-tfstate`. Para usar outro bucket, informe `-BucketEstado "NOME_DO_BUCKET"`. O script cria `terraform.tfvars` localmente quando ele ainda não existe; esse arquivo permanece ignorado pelo Git.

O parâmetro `-ConfirmarCustos` é obrigatório nas etapas que criam ou removem recursos. O modo `planejar` não executa `terraform apply`.

1. Autenticação

```
gcloud auth login
gcloud auth application-default login
gcloud config set project SEU_PROJECT_ID
```

2. Infraestrutura

```
Set-Location infrastructure/terraform
Copy-Item terraform.tfvars.example terraform.tfvars
terraform init
terraform fmt -check
terraform validate
terraform plan -out=tfplan
terraform apply tfplan
```

O arquivo `terraform.tfvars` deve receber o projeto, a região e as opções desejadas. Ele contém configurações locais e não deve ser enviado ao Git.

3. Pipeline Batch

Na raiz do projeto:

```
.\cloud\gcp\dataproc\submit-batch.ps1 `
  -Projeto "SEU_PROJECT_ID" `
  -Bucket "SEU_BUCKET" `
  -ContaServico "SUA_SERVICE_ACCOUNT" `
  -Executar
```

O job lê as fontes públicas do BigQuery, grava Bronze, Silver e Gold no Cloud Storage e permite o registro das três tabelas Gold no BigQuery.

4. Pipeline Streaming

1. publicar a imagem do produtor pelo Cloud Build;
2. habilitar Kafka, produtor e consumidor nas variáveis Terraform;
3. aplicar o plano revisado;
4. enviar o job PySpark com `cloud/gcp/dataproc/submit-streaming.ps1`;
5. validar logs e registros no Cloud Storage;
6. desativar os recursos pagos após a demonstração.

Detalhes adicionais estão em `[cloud/gcp/README.md](cloud/gcp/README.md)` e `[infrastructure/terraform/README.md](infrastructure/terraform/README.md)`.

Histórico de desenvolvimento com Git

O desenvolvimento foi dividido em funcionalidades para evidenciar a evolução do projeto. Foram utilizadas branches específicas para dados brutos, camadas Bronze, Silver e Gold, pipelines Batch, Streaming, orquestração, infraestrutura, qualidade, observabilidade, FinOps e documentação.

Cada etapa foi registrada com mensagens descritivas e integrada à `main` por Pull Request. Essa organização permite revisar as alterações, acompanhar a participação dos colaboradores e entender como a solução evoluiu desde a exploração inicial até a validação em nuvem.

Resultado

O projeto entrega uma pipeline híbrida funcional e reproduzível no Google Cloud. As seis fontes educacionais são processadas pelas camadas Bronze, Silver e Gold; as regras de qualidade impedem a publicação de dados críticos; os indicadores finais ficam preparados para análises, dashboards e futuras aplicações de IA. A infraestrutura, o monitoramento e os controles de custo são declarados como código e podem ser recriados em outro projeto GCP.